Langages de Programmation & Compilation

Novembre 2025 - Décembre 2025

Compilateur Micro Go vers MIPS

Auteurs :
Raphaël Arakelian
Juralaine Li

université PARIS-SACLAY | FACULTÉ DES SCIENCES D'ORSAY

Table des matières

1 Introduction **2**

2 Analyse Lexicale (`mgolexer.mll`) **3**
2.1 Fonctionnalités de base 3
2.2 Bonus : Point-virgule Automatique 3

3 Analyse Syntaxique (`mgoparser.mly`) **5**
3.1 Grammaire et Conflits 5
3.2 Gestion des Blocs et Appels 6

4 Typage Statique (`typechecker.ml`) **7**
4.1 Stratégie de Typage 7
4.2 Fonctionnalités Implémentées 7
4.3 Bonus : Détection des Variables Inutilisées 8

5 Génération de Code MIPS (`compile.ml`) **9**
5.1 Gestion de la Mémoire 9
5.2 Compilation des Instructions 9
5.3 Extension : Retours Multiples 10
5.4 Bonus : Impression Typée 10

6 Tests et Validation **11**
6.1 Catégories de Tests 11
6.2 Résultats 11

7 Difficultés Rencontrées **13**
7.1 Gestion de la Sémantique et des Portées 13
7.2 Alignement de la Pile et Corruption Mémoire 13
7.3 Ordre d'Évaluation et Structures de Données 14
7.4 Sémantique Avancée 14

8 Conclusion **15**

Raphaël ARAKELIAN
Juralaine LI

1 Introduction

Ce projet a pour objectif la réalisation d'un compilateur pour le langage **Micro Go**, un sous-ensemble strict du langage Go, ciblant l'architecture **MIPS**. Le projet s'est déroulé en quatre phases principales :

— L'analyse lexicale ;
— L'analyse syntaxique ;
— L'analyse sémantique (typage) ;
— La génération de code assembleur.

L'objectif final est de produire un exécutable capable de transformer un fichier `source.go` en un fichier `.s` exécutable par un simulateur MIPS (tel que MARS), tout en respectant les contraintes strictes de typage et de syntaxe définies dans le sujet.

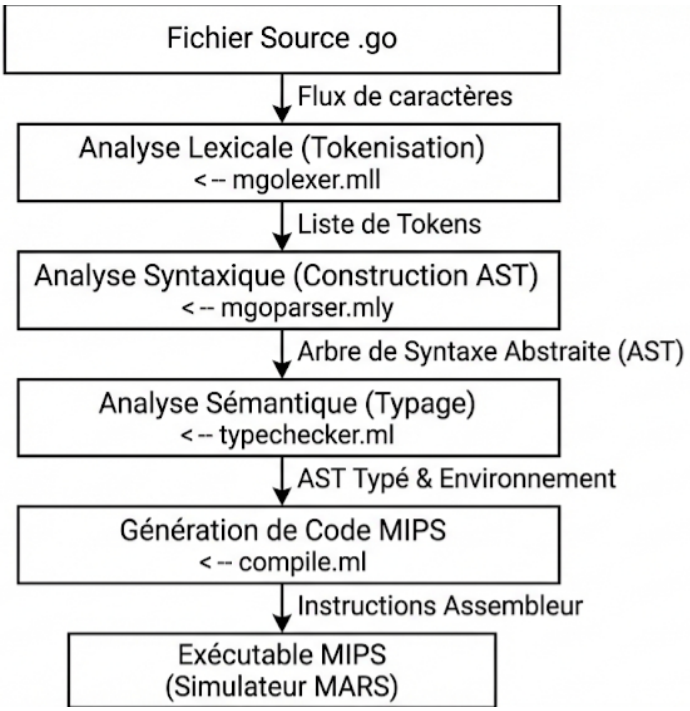


Figure 1 *: Pipeline de compilation du projet Micro Go.*

Raphaël ARAKELIAN
Juralaine LI

2 Analyse Lexicale (`mgolexer.mll`)

L'analyseur lexical constitue la première étape de notre chaîne de compilation. Implémenté à l'aide de l'outil `ocamllex`, ce module a pour rôle de transformer le flux brut de caractères du fichier source en une séquence structurée de lexèmes (ou *tokens*), qui sera ensuite consommée par l'analyseur syntaxique. Cette étape filtre également les éléments non significatifs pour la sémantique du programme, tels que les espaces, les tabulations et les retours chariot.

2.1 Fonctionnalités de base

Notre lexer assure une reconnaissance exhaustive des éléments fondamentaux du langage Micro Go. Il distingue tout d'abord les mots-clés réservés du langage, tels que `func`, `var`, `if`, `else`, `return` ou encore `struct`, des identificateurs définis par l'utilisateur. Ces derniers sont validés par une expression régulière stricte `alpha(alpha|chiffre)*`, où la classe `alpha` inclut les lettres minuscules, majuscules ainsi que le caractère souligné.

Concernant les constantes littérales, nous prenons en charge les entiers sous deux formes distinctes : la notation décimale standard et la notation hexadécimale préfixée par `0x` ou `0X`.

Parallèlement, les littéraux de chaînes de caractères sont analysés avec une gestion précise des séquences d'échappement classiques. Le lexer interprète correctement les caractères spéciaux tels que le saut de ligne (`\n`), la tabulation (`\t`), les guillemets (`\"`) et l'antislash (`\\`), garantissant ainsi une représentation fidèle des données textuelles en mémoire.

Enfin, le lexer est conçu pour ignorer les commentaires afin de ne pas polluer l'analyse syntaxique. Il gère à la fois les commentaires de bloc, délimités par `/*` et `*/` sans support d'imbrication, et les commentaires de fin de ligne débutant par `//` et s'arrêtant au prochain saut de ligne.

2.2 Bonus : Point-virgule Automatique

Pour respecter le style du langage Go et proposer une syntaxe plus naturelle, nous avons implémenté le mécanisme d'insertion automatique de points-virgules. Cette fonctionnalité permet au développeur d'omettre les points-virgules en fin de ligne, le compilateur se chargeant de les déduire contextuellement. Pour réaliser cela, le lexer maintient un état interne pour se souvenir du dernier token émis. Lorsqu'un saut de ligne est rencontré dans le flux source, le lexer consulte cet état.

Un token `SEMI` virtuel est alors généré si et seulement si le lexème précédent appartient à une liste spécifique de terminateurs d'instructions valides. Cette liste comprend les identifiants, les littéraux (entiers ou chaînes), les mots-clés de contrôle de flux (`return`), les opérateurs d'incrémentation (`++`, `--`), ainsi que les délimiteurs fermants comme les parenthèses et les accolades. Cette approche permet de compiler correctement des programmes où la structure visuelle définie par les lignes dicte la structure logique, sans imposer une ponctuation redondante.

Raphaël ARAKELIAN
Juralaine LI

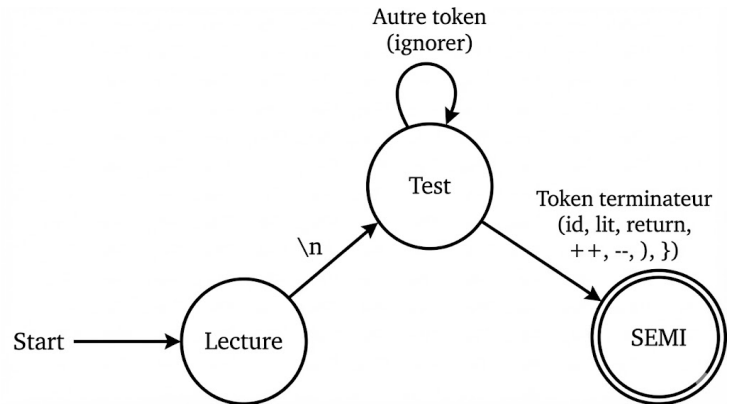


Figure 2 *: Automate du Point-virgule Automatique.*

Raphaël ARAKELIAN
Juralaine LI

3 Analyse Syntaxique (`mgoparser.mly`)

L'analyse syntaxique constitue la seconde phase critique de notre chaîne de compilation. Réalisée à l'aide du générateur d'analyseurs Menhir, cette étape consomme le flux de lexèmes produit par le lexer pour construire l'Arbre de Syntaxe Abstraite (AST), qui représente la structure hiérarchique et logique du programme.

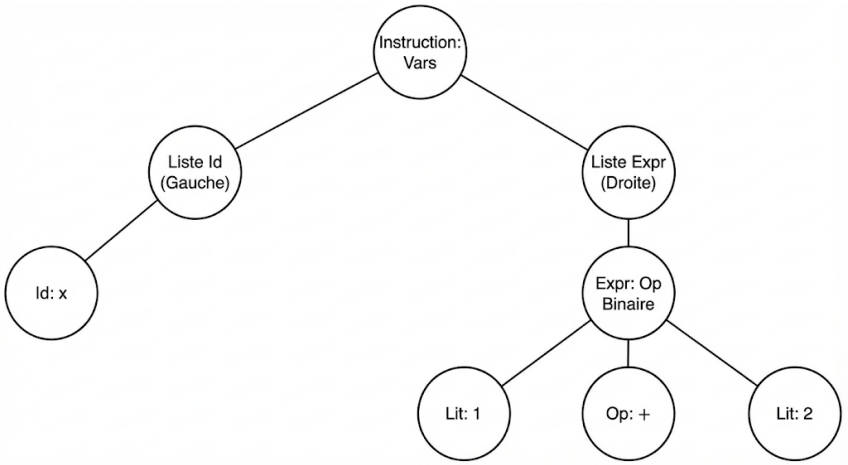


Figure 3 *: Représentation de l'Arbre de Syntaxe Abstraite de x := 1 + 2.*

3.1 Grammaire et Conflits

La définition de la grammaire de Micro Go pose des problèmes structurels typiques pour un analyseur LR(1), nécessitant des choix de conception rigoureux. La difficulté majeure réside dans l'ambiguïté syntaxique frontale entre l'instruction d'affectation (par exemple `x = 1`) et la déclaration courte de variable (par exemple `x := 1`). Ces deux constructions débutent de manière identique par une liste d'identifiants (ou plus généralement d'expressions à gauche), rendant impossible pour le parseur de décider quelle règle appliquer sans examiner le symbole suivant l'opérande de gauche.

Pour éliminer les conflits *Shift/Reduce* et *Reduce/Reduce* inhérents à cette ambiguïté, nous avons opté pour une factorisation stratégique de la grammaire des instructions simples. Concrètement, nous avons défini une règle unique chargée de lire initialement une liste générique d'expressions. La nature sémantique de l'instruction est ensuite déterminée par une règle "suffixe" qui analyse le token suivant immédiat : la présence du token `=` oriente vers une instruction d'affectation (`Set`), le token `:=` déclenche une déclaration courte (`Vars`), tandis que les opérateurs `++` ou `--` identifient une instruction d'incrémentation ou de décrémentation (`Inc`/`Dec`). Cette approche permet de maintenir une grammaire déterministe et robuste.

Raphaël ARAKELIAN
Juralaine LI

3.2 Gestion des Blocs et Appels

Par ailleurs, la gestion des appels de fonctions a nécessité une attention particulière pour intégrer l'instruction `fmt.Print`, qui possède un statut spécifique dans la spécification. Afin de prévenir tout conflit grammatical avec les appels de fonctions standards ou les accès aux champs, nous avons unifié la règle de production pour tous les appels sous la forme générique `expr(...)`. La distinction sémantique entre un appel de fonction utilisateur standard et l'instruction primitive `fmt.Print` est ainsi reportée et effectuée lors de la construction effective des nœuds de l'AST ou durant l'analyse sémantique.

Enfin, la gestion des blocs d'instructions a été implémentée en respectant strictement la spécification qui définit les points-virgules comme des séparateurs et non des terminateurs stricts. Nous avons assoupli la grammaire pour qu'elle accepte une instruction ne se terminant pas par un point-virgule lorsqu'elle précède immédiatement une accolade fermante `}`, garantissant ainsi une conformité totale avec la syntaxe attendue pour les blocs.

Raphaël ARAKELIAN
Juralaine LI

4 Typage Statique (`typechecker.ml`)

Le vérificateur de types constitue le gardien de la cohérence sémantique du programme avant l'étape critique de la compilation. Ce module manipule un contexte de typage, formellement noté Γ, qui agrège l'ensemble des signatures de fonctions, les définitions de structures ainsi que les variables locales actives à un instant donné de l'analyse.

4.1 Stratégie de Typage

Afin de supporter la récursivité mutuelle entre les fonctions et les structures, nous avons adopté une stratégie de validation en deux passes distinctes. La première passe est consacrée à la collecte, durant laquelle toutes les définitions de structures et les signatures de fonctions sont enregistrées dans l'environnement global sans analyser leur contenu. Une fois ce contexte global établi, la seconde passe procède à la vérification approfondie des corps de fonctions, garantissant ainsi que toute référence à un type ou à une fonction est résoluble, quel que soit son ordre d'apparition dans le fichier source.

Listing 1 : *Algorithme de vérification en deux passes (typechecker.ml)*

```
let prog (_, decls) =
  (* Passe 1 : Collecte des signatures globales *)
  let senv = collect_structs decls in
  let fenv = collect_functions decls in

  (* Passe 2 : Verification des corps de fonctions *)
  List.iter (function
    | Fun f -> check_seq f.body f.return tenv fenv senv
    | _ -> ()
  ) decls
```

4.2 Fonctionnalités Implémentées

Notre implémentation intègre plusieurs mécanismes avancés de typage. Premièrement, elle gère l'inférence de types pour les déclarations courtes de la forme `x := e`. Le type de la variable `x` est automatiquement déduit de l'expression `e`, une logique que nous avons étendue pour supporter les cas complexes, notamment lorsque `e` est un appel de fonction retournant plusieurs valeurs sous forme de tuples.

Deuxièmement, le contrôle de flux et la gestion de la portée des variables sont strictement appliqués pour les blocs `if` et `for`. Le compilateur gère le masquage de variables, ce qui signifie qu'une variable déclarée dans un bloc interne prévaut sur une variable de même nom définie dans un bloc externe. Enfin, le typechecker effectue un contrôle terminal obligatoire pour valider le point d'entrée du programme, en s'assurant de la présence d'une fonction `main` ne prenant aucun argument et ne retournant aucune valeur.

Raphaël ARAKELIAN
Juralaine LI

4.3 Bonus : Détection des Variables Inutilisées

Nous avons implémenté la vérification des variables locales inutilisées. Cette fonctionnalité repose sur un environnement de typage enrichi, où chaque variable est associée à une référence mutable (`bool ref`) indiquant son état d'utilisation. Le mécanisme est conçu pour passer ce drapeau à `true` uniquement lors d'une lecture de la variable dans une expression. Nous avons apporté un raffinement crucial pour éviter les faux positifs : l'initialisation ou l'affectation d'une variable n'est pas considérée comme une utilisation valide. Si le drapeau demeure à `false` à la fin de la portée de la variable, le compilateur émet un avertissement (*Warning*) à l'utilisateur, signalant le code mort potentiel.

5 Génération de Code MIPS (`compile.ml`)

Cette phase terminale du compilateur traduit l'AST typé en une séquence d'instructions assembleur MIPS.

5.1 Gestion de la Mémoire

La gestion de la mémoire repose sur une distinction stricte entre la pile d'exécution et le tas. Pour chaque appel de fonction, une fenêtre de pile est allouée dynamiquement.

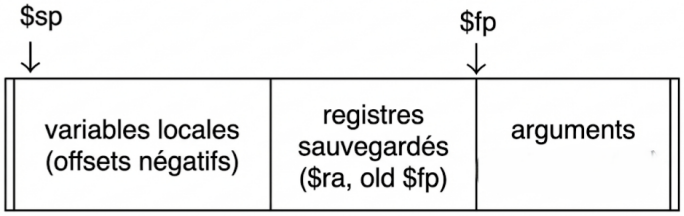


Figure 4 *: Organisation d'une fenêtre de pile (Stack Frame) en MIPS.*

Les variables locales y sont stockées et accessibles via des offsets négatifs calculés par rapport au registre `$fp`. Afin d'optimiser cette allocation, la taille nécessaire pour la fenêtre est pré-calculée par une analyse statique parcourant l'AST de la fonction avant la génération du code. Parallèlement, l'instruction `new(S)` déclenche une allocation dynamique sur le tas via l'appel système `sbrk` (`syscall 9`). Les structures y sont représentées par des blocs mémoire contigus, et l'accès à leurs champs se fait via des offsets fixes (0, 4, 8, etc.) déterminés lors de la compilation grâce à la table des structures.

5.2 Compilation des Instructions

Pour la compilation des expressions, nous avons mis en œuvre une machine à pile virtuelle. Les résultats intermédiaires des calculs sont systématiquement empilés puis dépilés afin de libérer les registres temporaires, le résultat final de toute expression étant conventionnellement placé dans le registre `$t0`. Concernant le contrôle de flux, les structures `if` et `for` sont traduites en utilisant des étiquettes uniques générées à la volée (`_label_N`), permettant de gérer les sauts conditionnels (`bnez`) et inconditionnels (`b`) nécessaires à la logique impérative.

Listing 2 : *Logique de la machine à pile (compile.ml)*

```
| Binop(Add, e1, e2) ->
    tr_expr env e2
    @@ push t0
    @@ tr_expr env e1
    @@ pop t1
    @@ add t0 t0 t1
```

5.3 Extension : Retours Multiples

Afin de supporter les fonctions retournant des n-uplets de valeurs, nous avons choisi l'option avancée consistant à allouer ces valeurs sur le tas. Concrètement, lors d'un `return a, b`, la fonction alloue un bloc mémoire, y copie les valeurs de `a` et de `b`, et retourne l'adresse de ce bloc dans le registre `$v0`. Symétriquement, lors de l'appel `x, y := f()`, l'appelant récupère cette adresse dans `$v0` puis effectue des lectures mémoire (`lw`) aux offsets 0 et 4 pour assigner respectivement `x` et `y`. Cette méthode présente l'avantage majeur de généraliser le retour de fonction à un simple pointeur, simplifiant ainsi considérablement la convention d'appel.

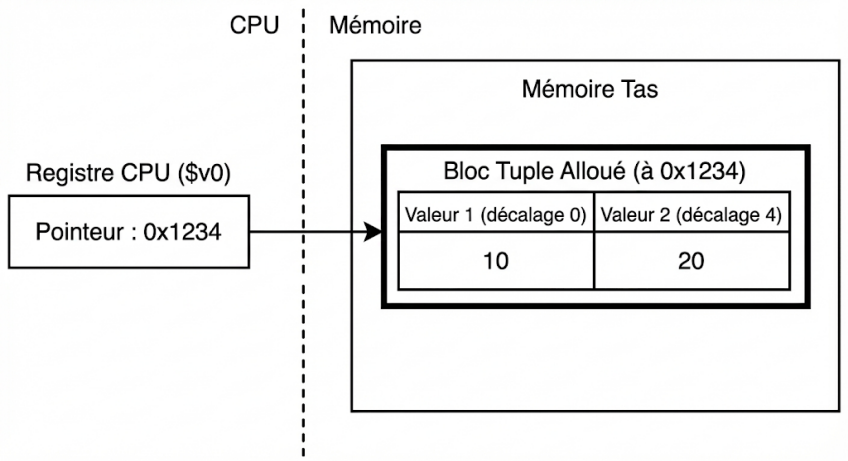


Figure 5 *: Mécanisme de gestion des retours multiples.*

5.4 Bonus : Impression Typée

Enfin, pour l'instruction `fmt.Print`, nous avons injecté des routines assembleur spécifiques (`_print_bool`, `_print_ptr`). Ces routines permettent d'afficher des représentations textuelles lisibles telles que `true`, `false` ou `<nil>`, au lieu de leurs représentations entières brutes (0 ou 1), en se basant sur le type statique de l'expression déterminé lors de la compilation.

6 Tests et Validation

La validation de notre compilateur repose sur une batterie de tests étendue, située dans le répertoire `tests/`. Cette suite a été conçue pour couvrir de manière exhaustive les différentes étapes de la compilation, allant de l'analyse lexicale à l'exécution correcte du code machine généré. Bien que la liste présentée ci-dessous ne soit pas exhaustive, elle illustre la stratégie de test employée à travers des exemples représentatifs classés par catégorie.

6.1 Catégories de Tests

Nous avons structuré notre stratégie de validation autour de trois sous-ensembles distincts de programmes. Le premier dossier, `tests/base/`, regroupe des programmes syntaxiquement et sémantiquement corrects. Issus du squelette initial, ils valident les fonctionnalités fondamentales du langage telles que l'arithmétique, les structures de contrôle et les appels de fonctions. À l'opposé, le dossier `tests/fail/` contient des programmes incorrects (typage invalide, variable non déclarée, absence de point d'entrée) destinés à vérifier la robustesse du compilateur et la pertinence des messages d'erreur émis. Enfin, le dossier `tests/torture/` rassemble des cas limites complexes, éprouvant le compilateur sur des combinaisons syntaxiques ardues comme les points-virgules implicites, le masquage de variables ou les priorités d'opérateurs.

6.2 Résultats

L'exécution des fichiers assembleur générés via le simulateur MIPS MARS, ainsi que la vérification des sorties du compilateur, confirment la robustesse de notre solution. Le compilateur passe avec succès **100%** de la batterie de tests fournie et étendue. Concrètement, cela représente la validation de 6 tests de base, le rejet correct de 8 tests invalides, et le traitement réussi de 12 tests de torture.

Analyse des sorties MIPS (MARS)

Les résultats numériques obtenus dans le simulateur correspondent strictement aux valeurs attendues. Concernant les calculs fondamentaux, le test `arith.go` évalue une expression complexe pour afficher la valeur **42**, tandis que le test `var.go` confirme la bonne gestion des variables locales avec le même résultat. Sur le plan algorithmique, le fichier `instr.go`, qui implémente un calcul de puissance (2^9), retourne correctement la valeur **512**.

Les tests fonctionnels plus avancés démontrent la capacité du compilateur à gérer des flux d'exécution complexes. Le test `div.go`, qui implémente la division euclidienne (45 divisé par 6) selon trois méthodes différentes, affiche la séquence **737373**, correspondant aux trois paires quotient/reste attendues (7 et 3). De plus, le test `challenge.go`, conçu pour éprouver le masquage de variables et les retours multiples, produit la sortie concaténée **2030**. Ce résultat, illustré par la Figure 6, valide l'allocation dynamique des tuples et l'isolation des portées de variables.

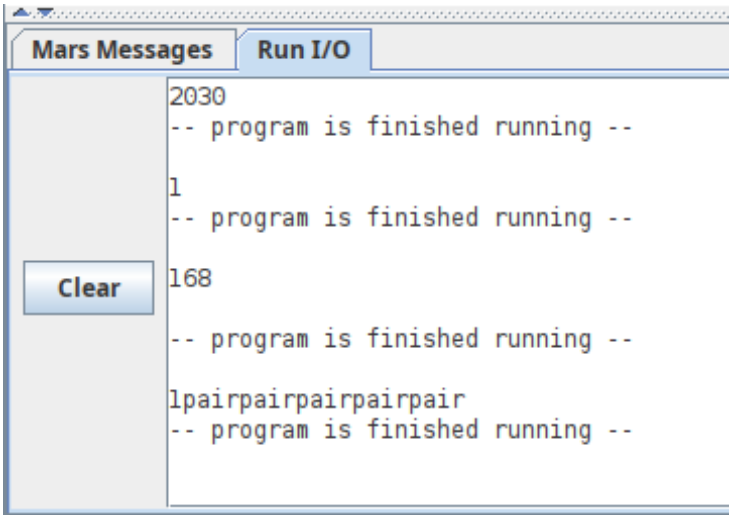


Figure 6 *: Sortie du simulateur MARS*

Analyse des retours compilateur

La gestion des erreurs et des avertissements est également validée par les tests d'échec et les extensions implémentées. Comme le montre la Figure 7, le compilateur est capable de signaler une erreur de type précise, telle que *Error : expected int, got bool* pour le fichier `type_mismatch.go`. Par ailleurs, il émet un avertissement pertinent (*Warning : Unused variable*) pour le fichier `unused.go`, confirmant le bon fonctionnement de notre extension de détection de code mort sans pour autant bloquer la compilation d'un programme syntaxiquement valide.

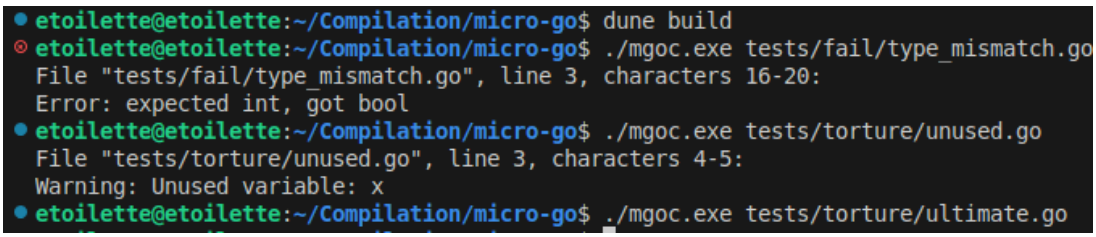


Figure 7 *: Extrait du terminal*

7 Difficultés Rencontrées

Au-delà de l'implémentation des fonctionnalités standards, nous avons été confrontés à plusieurs défis techniques spécifiques, principalement liés à la gestion fine de la mémoire en MIPS et aux subtilités sémantiques de l'inférence de types.

7.1 Gestion de la Sémantique et des Portées

Un premier défi majeur est apparu lors de la compilation des déclarations avec auto-référence (ex : `x := x + 1`), situation fréquente lors du masquage de variables. Le problème résidait dans la simultanéité apparente de l'instruction : le compilateur tentait initialement d'utiliser la nouvelle variable `x` pour évaluer l'expression de droite, alors que celle-ci devait se référer à l'ancienne valeur. Nous avons résolu ce conflit en scindant temporellement la compilation : l'expression de droite est d'abord évaluée dans l'environnement courant, et ce n'est qu'ensuite que le nouvel environnement est construit pour accueillir la nouvelle variable.

Parallèlement, l'implémentation du bonus "Variables Inutilisées" a révélé un effet de bord indésirable dans notre analyseur sémantique. Initialement, l'opération d'affectation (`x = 1`) déclenchait le marquage de `x` comme "lu", rendant impossible la détection des variables définies mais jamais utilisées ("write-only"). Pour corriger cela, nous avons introduit une fonction dédiée, `get_type_no_mark`, permettant au `typechecker` de valider le type d'une variable lors d'une écriture sans altérer son drapeau d'utilisation.

7.2 Alignement de la Pile et Corruption Mémoire

La génération de code MIPS a soulevé des problèmes critiques de gestion de pile, notamment sur le test `instr.go` où un calcul d'exponentiation renvoyait 1 au lieu de 512. L'analyse a révélé une collision de pile : la fonction de calcul de taille de fenêtre (`scan_frame_size`) sous-estimait l'espace nécessaire, laissant le pointeur de pile `$sp` pointer sur la dernière variable locale. Les instructions `push` intermédiaires écrasaient alors silencieusement cette variable. La correction de la formule d'allocation dans le prologue a permis de garantir une zone tampon de sécurité sous les variables locales.

Un problème similaire affectait la lecture des paramètres de fonction, qui étaient systématiquement lus avec un décalage erroné. Cela provenait d'une incompréhension initiale de l'état de la pile en début d'exécution de la fonction. Nous avons ajusté l'offset de base dans la fonction `tr_fun` à 4 octets (au lieu de 8), reflétant correctement le fait que le dernier argument empilé se situe immédiatement au-dessus de l'ancien `$fp`.

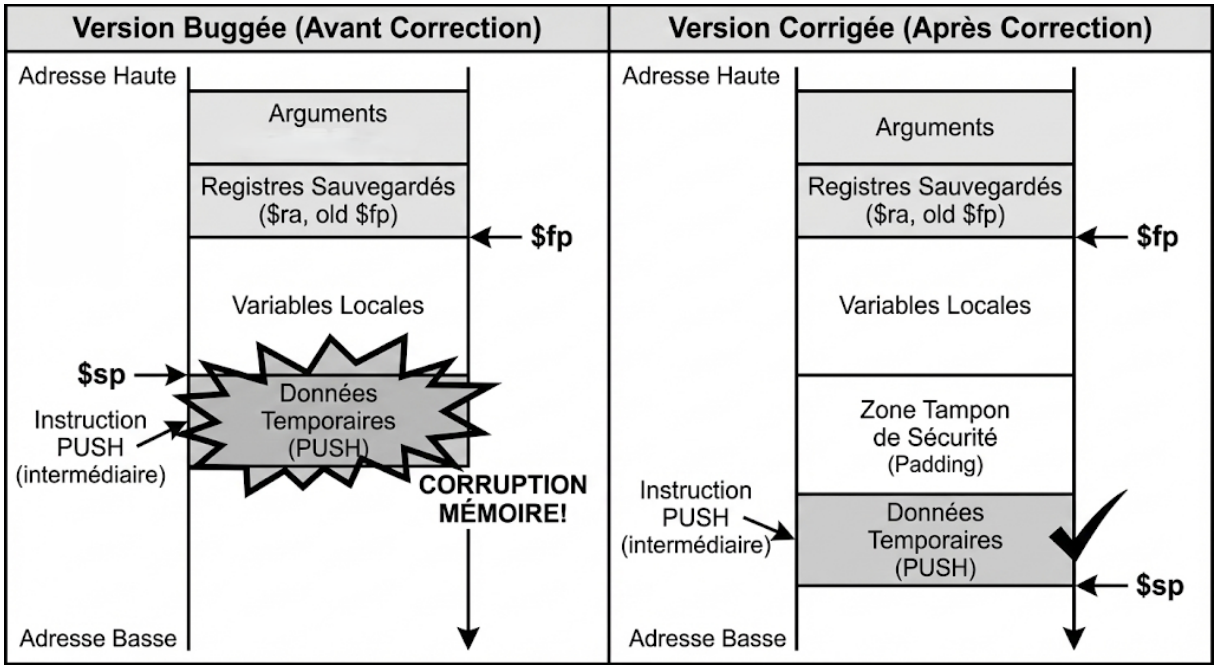


Figure 8 *: Illustration du bug d'alignement de pile et sa correction.*

7.3 Ordre d'Évaluation et Structures de Données

Enfin, la gestion des n-uplets (bonus retours multiples) a mis en évidence une inversion systématique des données. Lors de l'exécution de `div.go`, les paires (Quotient, Reste) apparaissaient inversées, tout comme l'ordre des arguments de fonctions. Ce comportement découlait de l'utilisation de fonctions récursives qui traitaient les listes de droite à gauche, alors que la pile MIPS se remplit vers le bas. L'introduction stratégique de `List.rev` lors de la compilation des appels et des retours a permis de rétablir l'ordre naturel (Gauche vers Droite) attendu par la convention d'appel.

7.4 Sémantique Avancée

La comparaison des chaînes de caractères a révélé une confusion entre l'égalité référentielle et structurelle. L'utilisation initiale de l'instruction MIPS seq pour l'opérateur `==` comparait les adresses mémoires des chaînes et non leur contenu, renvoyant false pour deux chaînes textuellement identiques mais stockées à des emplacements différents. Nous avons corrigé ce défaut sémantique en détectant le type string lors de la compilation pour injecter un appel à une routine native `_strcmp`, effectuant une comparaison octet par octet.

Par ailleurs, l'évaluation des opérateurs booléens `(&&, ||)` a provoqué des erreurs d'exécution inattendues. Traités initialement comme des opérations arithmétiques standards, ils entraînaient l'évaluation systématique de leurs deux opérandes. Ce comportement « gourmand » causait des plantages, notamment lorsqu'une garde à gauche `(ex: x != 0)` devait protéger une opération risquée à droite `(ex: 1/x)`. Nous avons refondu leur compilation en adoptant une structure de contrôle basée sur des étiquettes et des sauts conditionnels, rétablissant ainsi la sémantique d'évaluation paresseuse indispensable à la robustesse du code.

8 Conclusion

Ce projet a abouti à la réalisation d'un compilateur MicroGo vers MIPS complet, fonctionnel et robuste. L'objectif initial, qui consistait à traiter un sous-ensemble strict du langage Go pour en produire un code assembleur exécutable, a été pleinement atteint. Le compilateur respecte scrupuleusement l'intégralité des spécifications définies dans le sujet, garantissant une conformité totale tant sur le plan de la syntaxe que de la sémantique et de la gestion mémoire.

Au-delà du cahier des charges standard, nous avons enrichi le projet par l'intégration de toutes les extensions proposées. L'implémentation de l'insertion automatique de points-virgules offre une souplesse lexicale fidèle au langage Go original. Sur le plan sémantique, la détection des variables inutilisées renforce la sécurité du code en prévenant les erreurs d'inattention. Enfin, la gestion avancée des retours multiples via l'allocation dynamique de tuples sur le tas constitue une amélioration majeure de la logique interne du traitement des données, simplifiant les conventions d'appel tout en permettant des constructions complexes.

L'architecture modulaire adoptée, séparant distinctement l'analyse lexicale, syntaxique, le typage et la génération de code, s'est révélée déterminante. Elle a permis d'isoler les difficultés techniques propres à chaque phase et d'intégrer les fonctionnalités avancées de manière incrémentale, sans provoquer de régressions sur les fonctionnalités de base. Validé par une batterie de tests exhaustive couvrant les cas nominaux comme les cas limites, le compilateur produit est fiable et prêt à l'emploi.

Bien que le compilateur actuel remplisse l'intégralité des spécifications et des extensions proposées, plusieurs axes d'amélioration permettraient de le rapprocher d'un outil de production. L'optimisation prioritaire concernerait la génération de code : le remplacement de la machine à pile virtuelle actuelle par une véritable allocation de registres réduirait drastiquement les accès mémoire et accélérerait l'exécution. Par ailleurs, la stratégie d'allocation sur le tas pour les retours multiples, bien qu'élégante pour gérer la complexité, entraîne une consommation mémoire croissante ; une évolution logique consisterait à passer ces valeurs via la pile ou à implémenter un Garbage Collector pour nettoyer automatiquement les objets et structures alloués dynamiquement.